

# UNIVERSITY OF BUEA

Faculty of Engineering and Technology · Department of Computer Engineering

## FINAL YEAR PROJECT REPORT

# StudyHub

*A Web-Based Peer-to-Peer Collaborative Learning Platform with  
Integrated Mobile Money Payments for University Students*

---

Submitted by

**Tazanu Stanley**

**Supervisor:** \_\_\_\_\_

**Academic Year:** 2025 / 2026

**Date of Submission:** 13 September 2026

In partial fulfilment of the requirements for the degree of  
Bachelor of Engineering in Computer Engineering

# Abstract

---

University students routinely hold the knowledge their peers need, yet that knowledge stays trapped in private messaging groups, personal hard drives and individual notebooks. A student who misses a lecture, is stuck on a problem set, or needs one-to-one help on a single topic has no dependable place to look. Existing alternatives each solve part of the problem and none of it well: institutional learning management systems distribute material downwards from lecturers but carry no peer-to-peer channel; global question-and-answer sites are strong on programming and weak on local curricula; commercial tutoring marketplaces price in foreign currency and settle by card, which excludes most students in Cameroon, where payment is overwhelmingly by mobile money.

This project designed, implemented and deployed **StudyHub**, a full-stack web application that consolidates four functions into a single authenticated platform: subject-based study groups with real-time messaging, a shared repository of course notes, a question-and-answer forum with voting and accepted answers, and a peer-tutoring marketplace with booking and payment. Payment is handled natively through MTN Mobile Money and Orange Money, the two instruments students actually hold.

The system was built as two independently deployed tiers. The client is a React 19 single-page application, installable as a Progressive Web Application and usable on a degraded network; the server is an Express 5 REST API backed by PostgreSQL through the Prisma ORM, with a Socket.IO channel for real-time messaging. The implementation comprises approximately 20,700 lines of source across 88 REST endpoints and a 25-table relational schema, and is deployed live on a three-service free-tier infrastructure.

Particular engineering attention was paid to three areas where the operating context imposed constraints a textbook design does not anticipate. First, **payment integrity**: mobile money settlement is asynchronous and unreliable, so the system resolves all prices server-side, grants entitlements idempotently, and runs a background reconciler that guarantees a successful charge eventually grants access even if the payer closes the browser. Second, **content protection**: paid material is stored under authenticated delivery and released only through a signed, expiring URL issued after an entitlement check, while free material is redirected to a CDN — a change that reduced a 3.19 MB download from a 300-second timeout to 31 seconds on the same connection. Third, **usability under constraint**: the client tolerates a sleeping free-tier backend by retrying only transport-level failures, and the interface is fully bilingual in English and French, with 1,228 verified translation keys per language.

The system was verified through static analysis, production build validation, an automated locale-consistency checker, and structured manual testing across both languages and both colour themes. The

work delivers a functioning, publicly deployed platform; it does not include an automated regression test suite or a formal user evaluation, and both are identified as the principal directions for further work.

---

**Keywords:** collaborative learning; peer-to-peer education; web application; React; Node.js; PostgreSQL; mobile money; Progressive Web Application; real-time communication; internationalisation; Cameroon.

# Table of Contents

---

## 1 Introduction

---

- 1.1 Background of the study
- 1.2 Statement of the problem
- 1.3 Aim and objectives
- 1.4 Research questions
- 1.5 Scope and delimitation
- 1.6 Significance of the study
- 1.7 Organisation of the report

## 2 Literature Review and Related Work

---

- 2.1 Theoretical basis: peer-assisted learning
- 2.2 Review of existing systems
- 2.3 Comparative analysis
- 2.4 Identified gap

## 3 Methodology

---

- 3.1 Development methodology
- 3.2 Requirements elicitation
- 3.3 Functional requirements
- 3.4 Non-functional requirements
- 3.5 Technology selection and justification
- 3.6 Development environment

## 4 System Design

---

- 4.1 Architectural design
- 4.2 Use-case model
- 4.3 Database design
- 4.4 Process design: payment sequence
- 4.5 Interface design

## 5 Implementation

---

- 5.1 Authentication and authorisation

- 5.2 Collaboration subsystem
- 5.3 Content subsystem and access control
- 5.4 Tutoring and payment subsystem
- 5.5 Quality gate and moderation
- 5.6 Internationalisation
- 5.7 Resilience and offline capability

## **6 Testing and Validation**

---

- 6.1 Testing strategy
- 6.2 Static analysis and build validation
- 6.3 Automated consistency checking
- 6.4 Functional test cases and results
- 6.5 Defects found and resolved
- 6.6 Threats to validity

## **7 Results and Discussion**

---

- 7.1 Objectives achieved
- 7.2 Measured outcomes
- 7.3 Discussion of design decisions
- 7.4 Constraints imposed by context

## **8 Conclusion and Future Work**

---

- 8.1 Conclusion
- 8.2 Limitations
- 8.3 Recommendations for future work

## **— References**

---

## **A Appendix A — API endpoint summary**

---

## **B Appendix B — Database schema summary**

---

## **C Appendix C — Configuration reference**

---

# Introduction

---

## 1.1 Background of the study

Higher education depends on more than the formal transmission of material from lecturer to student. A substantial part of what a student learns is acquired laterally: from a classmate who has already understood a derivation, from a set of notes taken in a lecture one happened to miss, from an argument worked through with peers the evening before an assessment. This lateral channel is well documented in the education literature as peer-assisted learning, and it is largely informal.

In practice, the informality is the problem. At a typical Cameroonian university the channel exists almost entirely inside private WhatsApp groups, personal laptops and paper notebooks. Material circulates, but it circulates within closed circles, disappears when a group is archived, and is undiscoverable by anyone outside it. A student in the same department, taking the same course, may have no route to a resource that another student produced a week earlier.

Two further constraints shape the local context. Bandwidth is expensive and intermittent, which makes heavyweight platforms impractical and makes any design that assumes a stable connection fragile in use. Payment is overwhelmingly by mobile money rather than by card, so any system that intends to move money between students must settle through MTN Mobile Money or Orange Money; a card-only checkout excludes the intended users almost entirely.

## 1.2 Statement of the problem

There is no single, accessible, locally appropriate platform on which university students can share academic resources, resolve specific academic difficulties, and obtain paid one-to-one help from their peers. The absence manifests in four concrete ways:

1. **Knowledge is siloed.** Notes and explanations exist but are trapped in private channels and personal storage, so they benefit a handful of people rather than a cohort.
2. **Help is unreliable and untracked.** A question asked in a chat group scrolls out of view within hours. There is no record of which answer was correct, no accumulation of a searchable body of resolved problems, and no incentive for the person who answered.
3. **Tutoring is informal and unsafe.** Peer tutoring is arranged by word of mouth with no verification of competence, no agreed rate, no schedule and no payment record, which exposes both parties.

4. **Existing platforms do not fit.** International alternatives assume card payment, foreign currency pricing, generous bandwidth and a curriculum that is not the local one.

### 1.3 Aim and objectives

**Aim.** To design, implement and deploy a secure, accessible web-based platform that consolidates peer-to-peer academic collaboration and paid peer tutoring for university students, using payment instruments appropriate to the local context.

The aim is decomposed into the following specific objectives:

| NO. | OBJECTIVE                                                                                                                                                  |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O1  | To analyse the requirements of student collaboration and identify the shortcomings of existing systems in the local context.                               |
| O2  | To design a scalable client–server architecture, together with a normalised relational data model, supporting collaboration, content sharing and commerce. |
| O3  | To implement authenticated study groups with real-time messaging, a shared note repository, and a question-and-answer forum with reputation.               |
| O4  | To implement a peer-tutoring marketplace with application review, availability scheduling and booking.                                                     |
| O5  | To integrate mobile money payment with verifiable settlement, ensuring a successful charge always grants the purchased entitlement.                        |
| O6  | To enforce access control on paid content such that it is unreachable without a recorded purchase.                                                         |
| O7  | To ensure the platform remains usable under constrained network conditions and on mobile devices.                                                          |
| O8  | To deliver the interface in both English and French, reflecting Cameroon's official bilingualism.                                                          |
| O9  | To deploy the system publicly and validate it against the stated requirements.                                                                             |

**Table 1.1** — Specific objectives of the project.

### 1.4 Research questions

1. What functions must a peer-to-peer academic platform provide in order to displace the informal channels students currently rely upon?

2. How can an asynchronous, failure-prone mobile money settlement process be integrated such that entitlement is granted exactly once and never lost?
3. How can purchased digital content be protected from unauthorised access while remaining inexpensive to deliver at scale?
4. What design measures allow a web application on free-tier infrastructure to remain usable over an intermittent connection?

## 1.5 Scope and delimitation

**In scope.** A responsive web application, installable as a Progressive Web Application, covering: account management with role-based access; study groups with real-time chat; upload, discovery and download of course notes, both free and paid; a question-and-answer forum with voting, accepted answers and reputation; a tutor directory with applications, administrative review, availability and booking; mobile money payment with receipts; an administrative console; and a bilingual interface.

**Out of scope.** Native mobile applications for Android or iOS; video conferencing within the platform (sessions are arranged, not hosted); automated assessment or grading; integration with any university's existing student information system; and card or bank payment instruments.

## 1.6 Significance of the study

For students, the platform converts a scattered informal practice into a durable, searchable resource, and makes it possible to obtain targeted help without a prior social connection to the person able to give it. For those who tutor, it creates a verifiable record of competence and a payment mechanism they can actually use, turning unpaid favours into modest income. Methodologically, the project contributes a documented treatment of two problems under-addressed in general web-development literature: reliable entitlement granting over an asynchronous mobile money rail, and delivery of protected digital goods on constrained infrastructure.

## 1.7 Organisation of the report

Chapter Two reviews peer-assisted learning and comparable systems, and identifies the gap addressed. Chapter Three sets out the development methodology, the elicited requirements and the justification for each technology selected. Chapter Four presents the architectural, data and process design. Chapter Five documents the implementation of each subsystem. Chapter Six describes the testing strategy, results and defects found. Chapter Seven discusses the outcomes against the objectives. Chapter Eight concludes and identifies further work.

# Literature Review and Related Work

---

## 2.1 Theoretical basis: peer-assisted learning

The pedagogical premise of this project is that students learn effectively from one another. Two established ideas support it. Vygotsky's notion of the zone of proximal development holds that a learner progresses furthest when assisted by someone whose understanding is only modestly ahead of their own — a description that fits a classmate more closely than it fits a lecturer. Constructivist accounts of learning further hold that knowledge is built by the learner rather than transferred intact, which implies that the act of explaining a concept benefits the explainer as much as the recipient.

Both ideas argue for a system that is symmetric: every participant is simultaneously a potential consumer and a potential producer of help. This principle shaped the design directly. There is no separate class of "content provider"; any student may upload notes, answer questions, or apply to tutor, and the reputation mechanism rewards contribution rather than consumption.

## 2.2 Review of existing systems

### 2.2.1 Institutional learning management systems

Platforms such as Moodle and Google Classroom are the standard vehicle for course administration. Their orientation is vertical: a lecturer publishes, students receive. Where discussion features exist they are typically per-course forums with weak discovery and no reputation. They are also institution-bound — a student cannot obtain help from a peer at another university, or in a subject outside their enrolled courses — and self-hosted deployments impose an administrative burden the students themselves cannot discharge.

### 2.2.2 General question-and-answer platforms

Stack Overflow and the wider Stack Exchange network demonstrate convincingly that voting, accepted answers and reputation produce a durable, searchable knowledge base. The mechanism is sound and this project adopts it. The limitation is coverage: these platforms are strongest in software engineering, apply strict topicality rules that exclude coursework-style questions, and hold essentially nothing specific to a Cameroonian curriculum. They also offer no mechanism for arranging sustained one-to-one help.

### 2.2.3 Commercial note-sharing services

Course Hero and Studocu aggregate student-generated study material at scale and confirm demand. Two characteristics make them unsuitable here. Pricing is in foreign currency with card settlement, which excludes the target users on payment grounds alone. More fundamentally, their economics reward volume of uploads rather than quality, and the resulting quality variance is a known criticism; this project responds with an explicit quality gate and human review for paid material (Section 5.5).

### 2.2.4 Online tutoring marketplaces

Preply, Wyzant and comparable services implement the marketplace pattern this project adopts for tutoring: profiles, verification, scheduling, booking and payment. They are designed for cross-border tutoring at international rates, settle by card, and take a substantial commission. None operates meaningfully in the Cameroonian student market at student-affordable rates.

### 2.2.5 Informal messaging groups

The incumbent, and the system any replacement must actually beat, is WhatsApp. Its advantages are decisive: universal adoption, negligible data cost, and zero onboarding friction. Its deficiencies are equally clear. Content is ephemeral and unsearchable; a question scrolls away within hours; files are lost when a device is replaced; there is no notion of a correct answer; membership is by invitation, so discovery is impossible; and there is no payment or accountability mechanism for tutoring.

## 2.3 Comparative analysis

| SYSTEM                    | PEER-TO-PEER | PERSISTENT & SEARCHABLE | QUALITY SIGNAL | TUTOR BOOKING | MOBILE MONEY | LOW BANDWIDTH |
|---------------------------|--------------|-------------------------|----------------|---------------|--------------|---------------|
| Moodle / Google Classroom | No           | Partial                 | No             | No            | No           | Partial       |
| Stack Exchange            | Yes          | Yes                     | Yes            | No            | No           | Yes           |
| Course Hero / Studocu     | Yes          | Yes                     | Weak           | No            | No           | No            |
| Preply / Wyzant           | Partial      | n/a                     | Yes            | Yes           | No           | No            |
| WhatsApp groups           | Yes          | No                      | No             | No            | No           | Yes           |
| <b>StudyHub</b>           | <b>Yes</b>   | <b>Yes</b>              | <b>Yes</b>     | <b>Yes</b>    | <b>Yes</b>   | <b>Yes</b>    |

**Table 2.1** — Comparison of existing systems against the capabilities required in the target context. Assessments are based on the publicly documented feature sets and payment options of each platform.

## 2.4 Identified gap

No reviewed system combines all six capabilities. The strongest individual mechanisms are well proven and are deliberately reused: the reputation and accepted-answer model from Stack Exchange, and the profile-schedule-book-pay marketplace pattern from the tutoring services. What is absent is their integration into one platform, settled through mobile money and engineered for an intermittent connection. That integration is the contribution of this work.

### DESIGN CONSEQUENCE

Because the incumbent is WhatsApp rather than a competing web platform, the design is judged against a low-friction, near-zero-cost baseline. This drove three decisions: the application is installable and works partly offline so it behaves like a native app; the first load is optimised and a sleeping backend is handled gracefully rather than surfacing as an error; and group chat is real-time, since a slower-than-WhatsApp chat would not be adopted regardless of its other merits.

# Methodology

## 3.1 Development methodology

An iterative and incremental methodology was adopted in preference to a sequential waterfall approach. The justification is that several requirements — particularly the behaviour of the mobile money provider under failure — could not be specified accurately in advance and were only discovered by building against the live service. A methodology that fixed the specification before implementation would have fixed it wrongly.

Development proceeded in increments, each producing a deployable system. The version history records 65 commits between 27 July and 11 September 2026, and the database migration sequence provides an independent record of how the model evolved:

| MIGRATION | NAME                              | INCREMENT IT ACCOMPANIED                                                                                                    |
|-----------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 0001      | init                              | Core schema: users, groups, notes, questions, tutors, transactions.                                                         |
| 0002      | session_credits                   | Prepaid tutoring packs, added once per-session payment proved cumbersome for repeat bookings.                               |
| 0003      | message_reactions_and_soft_delete | Chat reactions, and soft deletion so a removed message leaves a visible trace rather than silently altering a conversation. |
| 0004      | premium_note_review               | Human moderation queue for paid uploads, added after recognising that automated checks cannot assess correctness.           |
| 0005      | uploader_trust                    | Support for bypassing review once a contributor has an established record.                                                  |
| 0006      | contact_messages                  | Contact form persistence with an administrative inbox.                                                                      |

Table 3.1 — Schema migrations as a record of incremental development.

## 3.2 Requirements elicitation

Requirements were derived from three sources: observation of how students currently exchange academic material in informal channels; the feature analysis of comparable systems presented in Chapter Two, which established both a baseline of expected functionality and the gaps to be filled; and constraints imposed by the operating environment, namely the prevalence of mobile money, the cost and instability of bandwidth, the predominance of mobile devices, and the official bilingualism of Cameroon.

### METHODOLOGICAL LIMITATION

Requirements were not elicited through a formal structured instrument such as a distributed questionnaire or recorded interview programme, and no statistical characterisation of the user population was produced. The requirements presented below are therefore reasoned rather than surveyed. This is a genuine weakness of the methodology and is revisited in Section 6.6 and Section 8.2.

## 3.3 Functional requirements

| ID    | AREA     | REQUIREMENT                                                                                                 | PRIORITY |
|-------|----------|-------------------------------------------------------------------------------------------------------------|----------|
| FR-01 | Accounts | A user shall register with name, email and password, and authenticate to obtain a session.                  | Must     |
| FR-02 | Accounts | A user shall maintain a profile recording university, field of study, biography and photograph.             | Must     |
| FR-03 | Accounts | The system shall distinguish student, tutor and administrator roles and restrict operations accordingly.    | Must     |
| FR-04 | Groups   | A user shall create a study group and specify whether joining is open or requires approval.                 | Must     |
| FR-05 | Groups   | Members shall exchange messages in real time, with file attachments, replies, editing and emoji reactions.  | Must     |
| FR-06 | Groups   | A group owner shall approve or deny join requests and remove members.                                       | Should   |
| FR-07 | Notes    | A user shall upload a note with title, description, subject and tags, and browse and filter the repository. | Must     |
| FR-08 | Notes    | A user shall preview PDF and image notes within the application before downloading.                         | Should   |
| FR-09 | Q&A      | A user shall post a question with optional voice recording and images, and receive answers.                 | Must     |

| ID    | AREA     | REQUIREMENT                                                                                          | PRIORITY |
|-------|----------|------------------------------------------------------------------------------------------------------|----------|
| FR-10 | Q&A      | Users shall vote on questions and answers; the asker shall mark one answer as accepted.              | Must     |
| FR-11 | Q&A      | The system shall accumulate reputation from a user's contributions and display standing.             | Should   |
| FR-12 | Tutoring | A user shall apply to tutor, stating subjects, experience and hourly rate.                           | Must     |
| FR-13 | Tutoring | An administrator shall approve, reject or revoke a tutor application.                                | Must     |
| FR-14 | Tutoring | An approved tutor shall publish weekly availability; a student shall book an available slot.         | Must     |
| FR-15 | Payment  | A student shall pay by MTN Mobile Money or Orange Money, with the operator detected from the number. | Must     |
| FR-16 | Payment  | A completed payment shall grant the corresponding entitlement exactly once and issue a receipt.      | Must     |
| FR-17 | Payment  | A user shall abandon a payment in progress without risk of being charged without receiving access.   | Must     |
| FR-18 | Content  | Paid content shall be inaccessible to any user without a recorded purchase.                          | Must     |
| FR-19 | Content  | Paid uploads shall pass an automated quality check and human review before sale.                     | Should   |
| FR-20 | Platform | Users shall receive in-application notifications of events concerning them.                          | Should   |
| FR-21 | Platform | An administrator shall manage users, review queues and view platform statistics.                     | Must     |
| FR-22 | Platform | A visitor shall contact the operators through a public form protected against automated submission.  | Could    |

**Table 3.2** — Functional requirements, prioritised by MoSCoW classification.

### 3.4 Non-functional requirements

| ID     | ATTRIBUTE              | REQUIREMENT AND ACCEPTANCE CRITERION                                                                                                          |
|--------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Security — credentials | Passwords shall be stored only as a salted adaptive hash. Criterion: bcrypt at a work factor of at least 12; no plaintext in storage or logs. |
| NFR-02 | Security — transport   | All production traffic shall be encrypted. Criterion: HTTP requests redirected to HTTPS; HSTS asserted.                                       |
| NFR-03 | Security — access      | Authorisation shall be enforced server-side on every protected operation, never by hiding the interface alone.                                |
| NFR-04 | Security — abuse       | Authentication and payment endpoints shall be rate-limited. Criterion: at most 10 login attempts per address per 15 minutes.                  |
| NFR-05 | Integrity — pricing    | Monetary amounts shall never be accepted from the client. Criterion: every charge resolved from the database at the time of the transaction.  |
| NFR-06 | Reliability            | A successful charge shall always result in the entitlement being granted, including if the client disconnects during settlement.              |
| NFR-07 | Usability — mobile     | The interface shall be operable on a 360 px viewport without horizontal scrolling.                                                            |
| NFR-08 | Usability — network    | A transport failure caused by a sleeping server shall be recovered transparently and communicated, not surfaced as an error.                  |
| NFR-09 | Accessibility          | Text and interface contrast shall meet WCAG 2.1 level AA. Criterion: 4.5:1 for body text, 3:1 for interface components.                       |
| NFR-10 | Internationalisation   | The complete interface shall be available in English and French, with the choice persisted across visits.                                     |
| NFR-11 | Performance            | Note downloads shall not be proxied through the application server where the content is unprotected.                                          |
| NFR-12 | Maintainability        | The database schema shall be version-controlled and applied only through reviewed migrations.                                                 |
| NFR-13 | Portability            | The client shall be installable as a Progressive Web Application on mobile and desktop.                                                       |
| NFR-14 | Privacy                | Analytics shall not employ cookies or collect personally identifying data.                                                                    |

**Table 3.3** — Non-functional requirements with acceptance criteria.

### 3.5 Technology selection and justification

Each major technology was selected against the requirements above rather than by familiarity alone. The reasoning is recorded below, including the alternatives rejected.

| DECISION         | SELECTED               | ALTERNATIVES<br>CONSIDERED                | JUSTIFICATION                                                                                                                                                                                                                                                                                                                                  |
|------------------|------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Client framework | React 19 with Vite     | Vue, Angular, server-rendered templates   | A single-page application suits an interface with a persistent chat connection and frequent partial updates. React's ecosystem supplied mature libraries for every secondary need. Vite was chosen over a bundler-based toolchain for build speed and native ES module handling.                                                               |
| Styling          | Tailwind CSS v4        | CSS modules, styled-components, Bootstrap | Utility classes keep styling local to markup and avoid an unbounded stylesheet. Version 4's <code>@theme inline</code> was decisive: it keeps generated utilities bound to CSS custom properties, which is what makes the runtime light/dark switch possible without a rebuild (Section 4.5).                                                  |
| Server runtime   | Node.js with Express 5 | Django, Laravel, Spring Boot              | A single language across both tiers reduces context switching and permits shared validation logic. Node's event-driven model suits the workload, which is dominated by concurrent I/O rather than computation. Express 5 propagates asynchronous handler rejections to the error middleware automatically, removing a class of silent failure. |
| Database         | PostgreSQL             | MySQL, MongoDB                            | The domain is strongly relational — memberships, votes, bookings and entitlements are all join-table relationships requiring referential integrity. A document store would have required these invariants to be enforced in application code. PostgreSQL additionally provides native array columns, used for tags and images.                 |
| Data access      | Prisma ORM             | Raw SQL, Sequelize, TypeORM               | Declarative schema as the single source of truth, with generated migrations and a type-aware client. This directly satisfies NFR-12; hand-written SQL would have made schema evolution across 25 tables error-prone.                                                                                                                           |

| DECISION            | SELECTED      | ALTERNATIVES<br>CONSIDERED                      | JUSTIFICATION                                                                                                                                                                                                                                                        |
|---------------------|---------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Real-time transport | Socket.IO     | Raw WebSocket, SSE, polling                     | Provides room semantics, automatic reconnection and transport fallback. Rooms map naturally onto study groups. Raw WebSocket would have required reimplementing all three.                                                                                           |
| Payment             | MeSomb        | Direct MTN and Orange APIs, Stripe, Flutterwave | A single integration covering both operators. Direct integration would have required two separate agreements and two distinct settlement flows. Card processors were excluded as inapplicable to the user population.                                                |
| File storage        | Cloudinary    | Local disk, Amazon S3                           | Local storage was rejected because the chosen host provides an ephemeral filesystem that is discarded on redeployment. Cloudinary additionally provides CDN delivery, transformation, and an authenticated delivery mode used to protect paid content (Section 5.3). |
| Session management  | Stateless JWT | Server-side sessions                            | Avoids shared session state, permitting the API to scale horizontally and simplifying the WebSocket handshake. The accepted cost is the absence of immediate revocation, discussed in Section 8.2.                                                                   |
| Analytics           | Plausible     | Google Analytics                                | Cookieless and collecting no personal data, satisfying NFR-14 and removing the legal requirement for a consent gate.                                                                                                                                                 |

**Table 3.4** — Technology decisions with alternatives and justification.

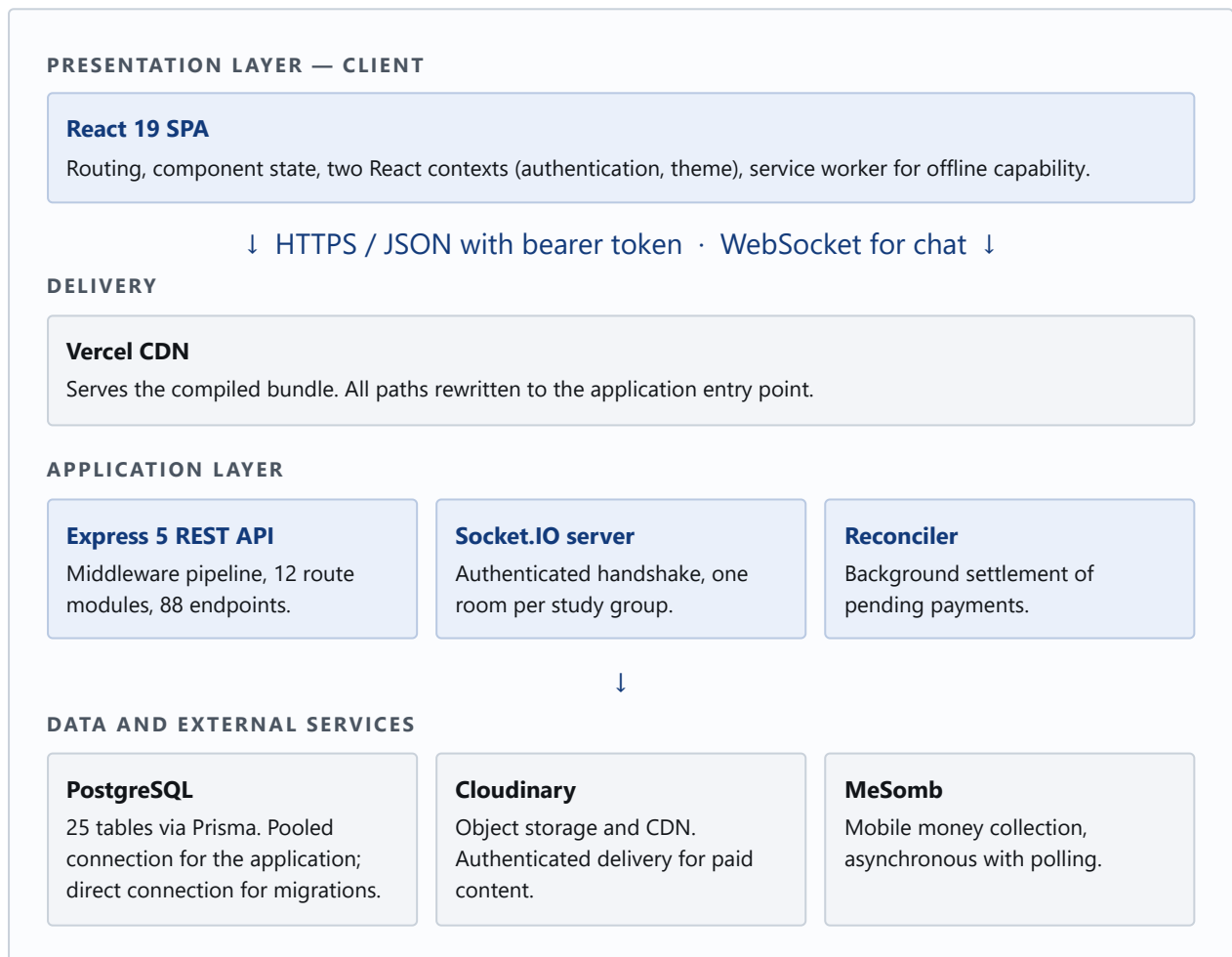
## 3.6 Development environment

Development was conducted on Windows 11 using Visual Studio Code, with Git for version control and a GitHub repository as the origin. Both application tiers reside in a single repository, from which both hosting providers deploy automatically on a push to the default branch. Quality tooling comprised ESLint for static analysis, the Vite production build as a compilation gate, and a purpose-written locale consistency checker described in Section 6.3.

# System Design

## 4.1 Architectural design

The system adopts a three-layer client-server architecture with the presentation layer deployed independently of the application layer. The separation is deliberate: static assets benefit from content-delivery distribution, whereas the application layer must hold long-lived WebSocket connections and run a background scheduler. These have incompatible hosting profiles, and the accepted cost of separating them is that the two tiers occupy different origins, making cross-origin policy a first-class design concern (Section 5.1).



**Figure 4.1** — System architecture. Each layer is deployed and scaled independently.

### 4.1.1 Request pipeline

Every authenticated request traverses a fixed middleware sequence. Ordering is significant and is enforced in the application entry point:

#### Request

|                     |                                                                     |
|---------------------|---------------------------------------------------------------------|
| -> trust proxy      | resolve true client address and protocol behind the host proxy      |
| -> HTTPS redirect   | 308, preserving method and body on non-GET requests                 |
| -> security headers | CSP, HSTS, nosniff, frame-ancestors none                            |
| -> CORS             | explicit origin allow-list; wildcard impermissible with credentials |
| -> body parser      | JSON, limited to 1 MB                                               |
| -> rate limiter     | selected by path                                                    |
| -> authenticate     | verify token, populate request identity                             |
| -> authorise        | role check, re-read from database                                   |
| -> handler          | Prisma query, response                                              |
| -> error handler    | log fully, return generic message                                   |

## 4.2 Use-case model

| ACTOR                                     | USE CASES                                                                                                                                                                                                                                                                                       |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Visitor</b><br>(unauthenticated)       | View landing and informational pages; register; authenticate; submit the contact form.                                                                                                                                                                                                          |
| <b>Student</b>                            | Manage profile; create, join and leave study groups; exchange group messages; upload, browse, preview and download notes; ask and answer questions; vote and accept answers; accumulate reputation; browse tutors; book a session; purchase a paid note; pay by mobile money; obtain a receipt. |
| <b>Tutor</b><br>(extends Student)         | Submit and maintain a tutor profile; publish weekly availability; accept, complete or decline bookings; publish paid notes subject to review.                                                                                                                                                   |
| <b>Administrator</b><br>(extends Student) | Review tutor applications; moderate the paid-note queue; suspend or promote users; adjust publishing trust; inspect platform statistics and payment diagnostics; read the contact inbox.                                                                                                        |
| <b>Payment provider</b><br>(external)     | Deliver the authorisation prompt to the payer's handset; report transaction status on request.                                                                                                                                                                                                  |

**Table 4.1** — Actors and their use cases. Tutor and Administrator both extend Student; a promotion adds capability without removing any.

## 4.3 Database design

The schema comprises 25 tables in third normal form, declared in a single Prisma schema file of 476 lines. Entities are grouped below by the subsystem they serve.

| SUBSYSTEM            | ENTITIES                                                                                                                       | PRINCIPAL RELATIONSHIPS                                                                                                               |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identity</b>      | users                                                                                                                          | Central entity, referenced by every other subsystem. Holds role, reputation and active state.                                         |
| <b>Collaboration</b> | groups , user_groups ,<br>group_join_requests ,<br>group_read_status                                                           | user_groups resolves the many-to-many relation between users and groups and carries the member's role within the group.               |
| <b>Messaging</b>     | group_messages ,<br>group_message_reactions                                                                                    | Messages belong to a group and an author, and may reference a parent message through a self-referential key implementing replies.     |
| <b>Content</b>       | notes                                                                                                                          | Belongs to an uploader and optionally to a group. Carries a premium flag and price.                                                   |
| <b>Knowledge</b>     | questions , answers ,<br>question_votes ,<br>answer_votes ,<br>question_bookmarks ,<br>question_followers ,<br>answer_comments | Votes are modelled as distinct entities rather than counters, so a uniqueness constraint makes double voting structurally impossible. |
| <b>Tutoring</b>      | tutors , tutor_availability ,<br>bookings , session_credits                                                                    | tutors is a one-to-one extension of a user carrying application status. A booking references both parties and a slot.                 |
| <b>Commerce</b>      | transactions ,<br>premium_notes ,<br>purchased_notes ,<br>premium_subscriptions                                                | transactions is the payment ledger;<br>purchased_notes is the entitlement record consulted before any protected file is released.     |
| <b>Platform</b>      | notifications ,<br>contact_messages                                                                                            | Notifications reference a recipient and the entity that triggered them.                                                               |

**Table 4.2** — Entity groups and principal relationships.

#### 4.3.1 Design decisions in the data model

- **Votes as entities.** Recording each vote as a row with a uniqueness constraint on voter and target makes repeat voting impossible at the storage layer. An integer counter would have pushed that guarantee into application code, where any path that forgets the check defeats it.
- **Soft deletion of messages.** A deleted message retains its row and is flagged, so the interface renders a placeholder rather than removing it. This preserves the coherence of a conversation others have already replied to.

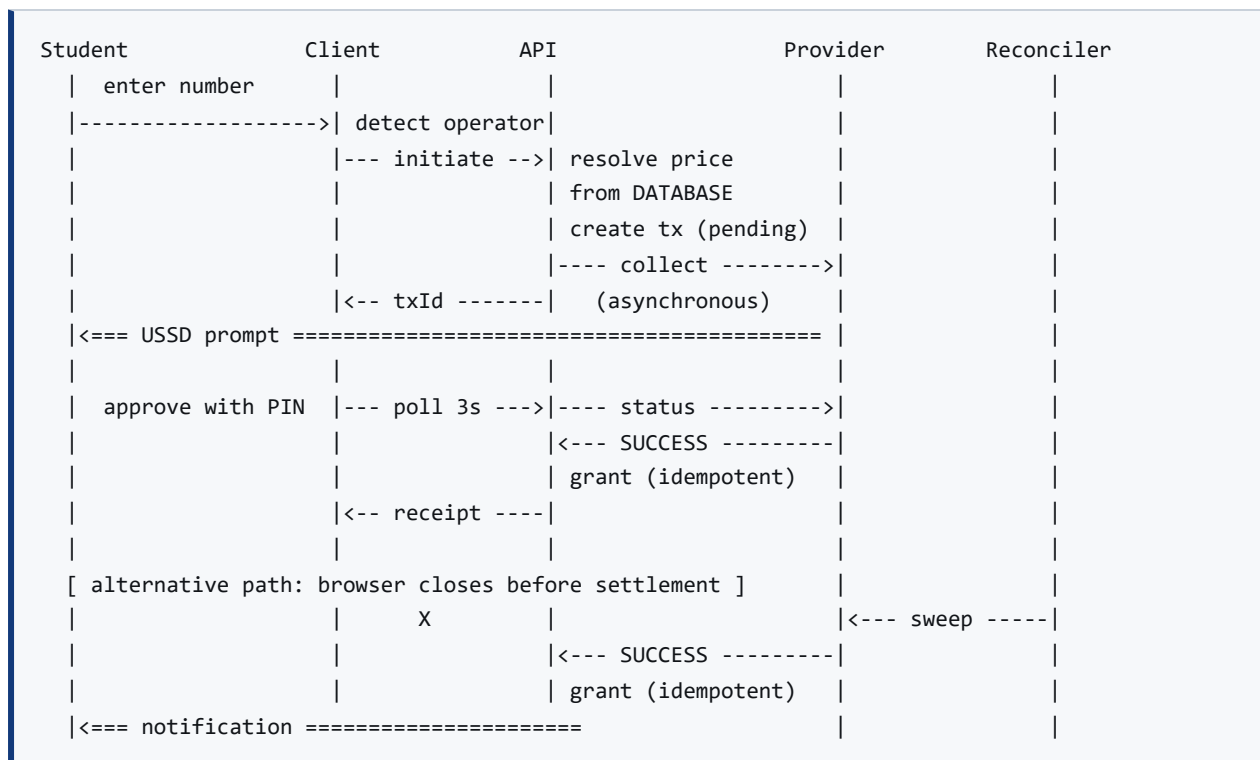
- **Entitlement as a record, not a flag.** Access to paid content is determined by the existence of a `purchased_notes` row rather than a boolean, yielding an auditable record of who purchased what and when.
- **Trust computed, not stored.** Permission to publish paid material without review is derived at query time from recent review history. A stored flag requires something to remember to revoke it; a derived one lapses automatically when the history no longer supports it.

### 4.3.2 Connection strategy

The datasource declares two connection strings. The application uses a pooled endpoint, keeping concurrent connections within the limits of the hosting tier. Migrations use a direct endpoint, because the pooler operates in transaction mode and cannot carry the session-scoped advisory locks and type-creation statements that migrations require. Routing migrations through the pooled endpoint produces a failure whose message does not identify pooling as the cause, which is recorded here because it cost material debugging time.

## 4.4 Process design: payment sequence

Payment is the most intricate process in the system, because the provider settles asynchronously and may never respond at all. The design therefore treats the browser as an observer of the outcome rather than as the authority on it.



**Figure 4.2** — Payment sequence, including the path taken when the client disconnects before settlement completes.

## TWO INVARIANTS THIS DESIGN PROTECTS

**Price integrity.** The client never transmits an amount. Every charge is resolved from the database when the transaction is created, so a modified request cannot alter what is charged.

**Exactly-once granting.** The browser poll and the reconciler may observe success concurrently. Granting is therefore performed through a conditional status update, so exactly one of the racing callers performs the grant and the other observes that it has already occurred.

## 4.5 Interface design

The interface is built on a token-based design system rather than per-page styling. One stylesheet defines a brand scale, neutral surfaces, semantic status colours, elevation and radii as CSS custom properties. Tailwind's `@theme inline` directive exposes these as utility classes that retain a `var()` reference instead of compiling to a fixed value, which is what permits the light and dark themes to switch at runtime by changing a single attribute on the document root, with no rebuild and no flash of restyled content.

Seventeen shared primitives implement every recurring visual element. Consolidating them replaced a situation in which the same component — a button, a modal, a status badge — had been implemented separately on each page and had drifted apart visually. Contrast ratios in both themes were chosen to satisfy NFR-09, which required separating the accent colour used for text from the one used as a fill behind white text, since a single value cannot satisfy both the 4.5:1 and 3:1 thresholds simultaneously.

The layout is mobile-first throughout, in recognition that the majority of the intended users will access the platform on a handset over a metered connection.

# Implementation

---

This chapter documents how each subsystem was realised, concentrating on decisions whose rationale is not evident from the code alone.

## 5.1 Authentication and authorisation

Passwords are hashed with bcrypt at a work factor of twelve, satisfying NFR-01. Authentication issues a JSON Web Token carrying the user identifier, signed with a server-held secret and expiring after twenty-four hours. The client stores the token and attaches it to every request through an interceptor, so no call site handles the header individually.

Two details in the login path are worth recording. First, a bcrypt comparison is executed even when the submitted email matches no account, comparing against a discarded dummy hash. Without this, a non-existent account returns measurably faster than an incorrect password, permitting an attacker to enumerate registered addresses by timing alone. Second, role checks re-read the role from the database rather than trusting the claim embedded in the token, so that a revoked administrator loses access immediately instead of at token expiry.

Three authorisation middlewares are provided. `authenticate` requires a valid token. `optionalAuth` populates the identity when a token is present but never rejects, serving endpoints that are public yet render differently for a signed-in user — for instance, indicating whether the viewer already owns a paid note. `requireRole` is a factory enforcing one or more roles. The WebSocket handshake reuses the same verification, and group membership is checked against the database before a socket is admitted to a room, so room membership cannot be asserted by the client.

Because the two tiers occupy different origins, cross-origin policy is load-bearing rather than incidental. Origins are supplied by configuration as an explicit allow-list; a wildcard is impermissible because credentials are enabled. Two diagnostics were added after misconfiguration proved difficult to trace: a rejected origin is logged by name alongside the allow-list, exposing a trailing slash or scheme mismatch that is otherwise invisible; and a production server booting with only a local origin configured prints a prominent startup error, because that state rejects every browser request with an opaque status that presents as a client-side defect.

## 5.2 Collaboration subsystem

Group messaging is delivered over Socket.IO, with one room per group. The server emits events for new messages, edits, deletions, reactions, typing state and presence. Message sending is optimistic: the message appears immediately in the sender's view marked as pending and is replaced when the server's copy arrives. The echo of the sender's own message is suppressed, since otherwise the author would see it twice.

A polling fallback is retained but deliberately subordinated. When the socket is connected the client polls every thirty seconds; when it is not, every four. The poll exists to repair events missed during a brief disconnection, not to drive the conversation. This matters on an unstable connection, where a socket may drop without the client observing a clean close.

Message editing is permitted within fifteen minutes of posting. Deletion is soft, as described in Section 4.3.1, and may be performed by the author or by a group administrator acting as moderator.

## 5.3 Content subsystem and access control

Uploads are parsed by Multer and streamed directly to Cloudinary without touching the application's filesystem, which is necessary because the hosting platform provides an ephemeral disk that is discarded on every redeployment. Files are constrained to 20 MB and validated against a MIME allow-list.

Delivery differs by whether the content is protected, and this distinction produced the single largest measured performance improvement in the project:

- **Free content** is not proxied. The server responds with a redirect to the content-delivery URL, so the file is served by the CDN rather than streamed through a constrained application process.
- **Paid content** is uploaded under Cloudinary's authenticated delivery type, which is not retrievable from a plain URL. After confirming that an entitlement record exists for the requesting user and the requested note, the server issues a short-lived signed URL. Legacy files held locally are stored beneath a directory that the static middleware refuses outright, so the only route to them is through the entitlement check.

### MEASURED OUTCOME (NFR-11)

Before the redirect was introduced, a 3.19 MB PDF proxied through the application server did not complete within the 300-second client timeout. After the change, the same file on the same connection was delivered in 31 seconds. The improvement is attributable to removing a constrained single process from the data path, not to any change in the file.

## 5.4 Tutoring and payment subsystem

All provider interaction is confined to a single adapter module, so that no route communicates with the payment SDK directly. The adapter normalises telephone numbers to a nine-digit local form, accepting the international and spaced variants students actually type; detects the operator from the prefix and rejects a mismatch between the detected operator and the selected service before any request is issued; and requests collection in asynchronous mode so the HTTP call returns immediately rather than blocking for the duration of the subscriber's interaction with the prompt.

Cancellation is not a local state change. When a payer abandons a payment, the server first interrogates the provider, because settlement may have completed between the decision to cancel and the arrival of the request. A reported success completes the order and grants access; a reported failure marks the transaction failed; an absent reference cancels, nothing having been charged; and a still-pending state is left pending and handed to the reconciler, with the user informed that confirmation is continuing. Without this, a user who cancelled a fraction of a second after approving would have been charged and left without access.

### DEFECT DISCOVERED DURING INTEGRATION

The provider authenticates each request with a signature computed over the serialised request body. Any character above U+007F in that body causes rejection with a message indicating a bad signature — an error that directs attention to the credentials rather than to the payload. A single typographic dash in a note title was sufficient to break every purchase of that note while purchases of other notes continued to succeed, which made the fault appear intermittent and unrelated to content.

The resolution was a sanitising helper in the adapter that folds typographic punctuation to ASCII equivalents, removes any remaining character above U+007F, collapses whitespace and truncates to 120 characters. Every field transmitted to the provider passes through it. The episode is recorded because the initial diagnosis was wrong: the credentials were suspected first, and a diagnostic endpoint had to be written to establish that the deployed keys were byte-identical to the working local ones before attention moved to the payload.

## 5.5 Quality gate and moderation

Charging for content creates an obligation to ensure it is worth paying for. Two mechanisms operate in sequence.

An automated gate examines each paid upload, extracting text from PDF documents by decompressing the relevant content streams and reading the text-showing operators, then measuring word count, page count, byte size and lexical variety. Variety is computed as a type-to-token ratio averaged over fixed windows rather than across the whole document, because an

unwindowed ratio falls as length rises and would therefore penalise long documents purely for being long.

The scope of this gate is deliberately limited, and the limitation is documented in the source. No heuristic can distinguish a correct derivation from a confidently incorrect one. What the gate detects is material that is obviously unfit for sale: near-empty files, placeholder text, padding and unreadable scans. Correctness is the province of the human review queue, and the automated stage exists only to keep the obviously unfit from reaching it.

Human review is performed by an administrator, who must open and read the file before deciding. A rejection requires a written reason, which is transmitted to the author, on the principle that a rejection without an actionable explanation merely wastes the contributor's effort. Contributors with an established record bypass the queue, using the computed trust rule described in Section 4.3.1: at least three approvals and no rejections within the last ten reviewed uploads.

## 5.6 Internationalisation

The interface is fully available in English and French, with 1,228 translation keys in each language, satisfying NFR-10. Language is detected in the order stored preference, then browser setting, then document attribute. A stored choice deliberately takes precedence over the browser, since a user who has explicitly selected a language should not have that selection overridden on a subsequent visit. Regional variants are mapped onto the base language, so that a browser reporting `fr-CM` resolves to French; without this mapping, Cameroonian browsers reporting a region would silently have received English, which is precisely the population the requirement exists to serve.

The governing implementation rule is that values which are stored or searched remain in English, and only their displayed labels are translated. Subjects, fields of study, availability days, question categories and booking statuses are all persisted. Translating the stored value would mean that a French-speaking tutor's subject never matched an English-speaking student's search for the same subject, and the two would silently fail to find one another.

Several consequences of this rule required specific handling. Plural forms are expressed through the internationalisation library's plural categories rather than by appending a character, because French pluralisation does not follow the English pattern. Dates and numbers are formatted in the active locale. The account-deletion confirmation compares typed input against the translated word, since translating only the visible instruction would have left French users typing the French word at a permanently disabled button. Transient notification text is resolved from the library instance rather than the React hook, which keeps the translation function out of data-fetching closures and their dependency arrays.

A consistency checker was written to prevent the two locale files from diverging as the application grows. It fails the build on four conditions: a key present in one language but absent from the other, differing interpolation placeholders between languages, a plural family missing a required form, and a key referenced in code but defined nowhere. It also reports keys identical in both languages and keys nothing references, without failing, since both are usually legitimate but merit inspection. Its value was demonstrated immediately, as described in Section 6.5.

## **5.7 Resilience and offline capability**

The hosting tier suspends the application server after approximately fifteen minutes without traffic, and the first request to reach a suspended instance is typically lost while the container starts. Untreated, a visitor arriving from a shared link encounters a failure on an entirely healthy service.

The client therefore retries, but narrowly. Only requests that received no response whatsoever are retried, at most twice and four seconds apart. Any request the server actually answered — including one answered with a server error — is not retried, because repeating a request the server did process risks duplicating its effect. While a retry is outstanding the client displays an explanatory banner, so the delay is accounted for rather than silent. This satisfies NFR-08.

The application is installable as a Progressive Web Application, satisfying NFR-13. Its service worker applies caching strategies chosen per resource class rather than uniformly: public statistics are served stale while revalidating; listings of groups, notes and questions prefer the network but fall back to cache, making them readable offline; and authentication, payment, messaging and tutor rates are never cached, because serving any of those from a stale copy would be actively harmful.

# Testing and Validation

---

## 6.1 Testing strategy

Validation combined four techniques: continuous static analysis, production build validation as a compilation gate, a purpose-written automated consistency checker, and structured manual functional testing. The strategy and its principal weakness are stated plainly below.

### PRINCIPAL WEAKNESS OF THE STRATEGY

The project does not include an automated regression test suite. There is no unit, integration or end-to-end test framework configured, and no test runner in either package manifest. Verification of behaviour was performed by ad-hoc scripts written to exercise a specific change and by manual execution of test cases; the scripts were not retained as a maintained suite. Consequently there is no automated protection against regression, and every change is re-verified by hand. This is the most significant deficiency of the work and is the first recommendation in Section 8.3.

## 6.2 Static analysis and build validation

ESLint runs across the client source with the React Hooks and React Refresh rule sets enabled. The codebase carries a known baseline of 42 errors and 13 warnings, predominantly effect-ordering advisories and unused bindings. The baseline was not eliminated; instead the working rule adopted was that no change may increase either count, which converts the tool into a usable regression signal despite the outstanding debt.

The production build is treated as the authoritative gate rather than the development server. This is a consequence of the styling approach: a mistyped design token resolves silently in development but fails the production build, so a change that runs correctly under the development server is not thereby validated.

## 6.3 Automated consistency checking

A checker was written to enforce internal consistency between the two locale files, invoked through a package script. It fails on four conditions, each of which would otherwise reach production as visibly incorrect text:

| CONDITION DETECTED                                  | SYMPTOM IF UNDETECTED                                                                      |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------|
| Key present in one language only                    | The interface falls back to the other language mid-page.                                   |
| Interpolation placeholders differ between languages | A placeholder renders literally, e.g. the text <code>{{count}}</code> appearing on screen. |
| Plural family missing a required form               | The key name itself is displayed instead of the text.                                      |
| Key referenced in code but undefined                | The raw key path is rendered to the user.                                                  |

**Table 6.1** — Conditions enforced by the locale consistency checker.

At final measurement the checker reported 1,228 keys in each language with no missing keys, no extra keys, no placeholder mismatches, no plural family defects and no undefined references.

## 6.4 Functional test cases and results

Functional verification was conducted against the deployed system, exercising each requirement of Table 3.2. A representative selection is reproduced below; results are as observed at final verification.

| ID   | REQ.   | TEST CASE AND EXPECTED RESULT                                                                        | RESULT |
|------|--------|------------------------------------------------------------------------------------------------------|--------|
| T-01 | FR-01  | Register with a valid address, then authenticate. Expect a session and redirection to the dashboard. | Pass   |
| T-02 | NFR-01 | Inspect the stored password field after registration. Expect a bcrypt hash, never plaintext.         | Pass   |
| T-03 | NFR-04 | Submit eleven failed logins within the window. Expect rejection after the tenth.                     | Pass   |
| T-04 | FR-05  | Post a message from one account; observe arrival in a second session without refresh.                | Pass   |
| T-05 | FR-05  | Delete a message that has been replied to. Expect a placeholder, with the reply chain intact.        | Pass   |
| T-06 | FR-07  | Upload a PDF note, then download it from a second account. Expect successful retrieval.              | Pass   |

| ID   | REQ.   | TEST CASE AND EXPECTED RESULT                                                                                                      | RESULT |
|------|--------|------------------------------------------------------------------------------------------------------------------------------------|--------|
| T-07 | FR-10  | Vote on the same answer twice from one account. Expect the second vote to be refused.                                              | Pass   |
| T-08 | FR-15  | Complete a purchase using an MTN number. Expect prompt, approval, entitlement and receipt.                                         | Pass   |
| T-09 | FR-17  | Cancel a payment in progress before approving. Expect cancellation with no charge.                                                 | Pass   |
| T-10 | NFR-06 | Close the browser after approving, before settlement. Expect the reconciler to grant access subsequently.                          | Pass   |
| T-11 | FR-18  | Request a paid note's file URL directly without a purchase record. Expect refusal.                                                 | Pass   |
| T-12 | NFR-05 | Submit a purchase request with a modified amount. Expect the server-side price to be charged.                                      | Pass   |
| T-13 | FR-13  | Approve a tutor application as administrator. Expect the tutor to appear in the directory.                                         | Pass   |
| T-14 | FR-19  | Submit a near-empty file as a paid note. Expect the quality gate to withhold it from sale.                                         | Pass   |
| T-15 | NFR-10 | Load the site with the browser reporting <code>fr-FR</code> . Expect French throughout and <code>lang="fr"</code> on the document. | Pass   |
| T-16 | NFR-10 | Select a language explicitly, then revisit with a conflicting browser language. Expect the stored choice to persist.               | Pass   |
| T-17 | NFR-07 | Operate every page at a 360 px viewport. Expect no horizontal overflow.                                                            | Pass   |
| T-18 | NFR-08 | Issue a first request to a suspended server. Expect transparent retry with an explanatory banner.                                  | Pass   |
| T-19 | NFR-02 | Request the production API over plain HTTP. Expect a redirect preserving method and body.                                          | Pass   |
| T-20 | FR-21  | Attempt an administrative endpoint as a student. Expect refusal on the server, not merely a hidden control.                        | Pass   |

**Table 6.2** — Representative functional test cases and observed results. Language and theme variants were exercised for each interface case.

## 6.5 Defects found and resolved

The defects below were discovered during development and verification. They are reported in full, including those arising from an incorrect initial diagnosis, since the detection mechanism is as instructive as the fault.

| DEFECT                                                   | DETECTED BY       | CAUSE AND RESOLUTION                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Every tutor application failed with a server error       | Manual testing    | The insert named a column that does not exist in the schema, the field having been renamed. Corrected to the schema name. The failure affected the route from its creation.                                                                                                                                            |
| Trust adjustment endpoint failed with a server error     | Manual testing    | A required module was never imported. Syntax checking passed because the reference is resolved only at execution. An automated test asserting on the status code of that call would have caught it; the ad-hoc check written at the time asserted on a subsequent call instead and reported a false pass.              |
| All purchases of one note rejected as a bad signature    | Manual testing    | A typographic dash in the note title placed a non-ASCII character in the signed request body. Resolved by sanitising all provider-bound fields. The credentials were incorrectly suspected first and a diagnostic endpoint had to be written to exclude them.                                                          |
| Notification panel rendered at 106 px on mobile          | Visual inspection | A fixed-position element resolved against an ancestor carrying a transform rather than against the viewport. Resolved by rendering through a portal. An initial automated assertion had reported the element as within the viewport, which was true and useless; the fault was only apparent from the rendered output. |
| Login controls and 404 suggestions remained untranslated | Locale checker    | Keys were defined but never referenced, the markup retaining literal English. Found by the checker's report of defined-but-unreferenced keys on pages believed complete.                                                                                                                                               |
| Build failure: duplicate identifier                      | Production build  | A component used the same short identifier for an interval handle, a data item and the translation function. Resolved by renaming. Two further instances of the same shadowing were found and corrected in other components.                                                                                           |
| Translation function referenced outside its component    | ESLint            | A child component called the function from its parent's scope. Reported as an undefined variable rather than shipping as a blank label.                                                                                                                                                                                |

| DEFECT                                                | DETECTED BY          | CAUSE AND RESOLUTION                                                                                                                                                                                             |
|-------------------------------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deployment rejected by the hosting platform           | Deployment           | Comment-style keys had been placed in a configuration file validated against a strict schema. Removed.                                                                                                           |
| Duplicate meta description tags                       | Inspection of output | The framework de-duplicates title elements but not meta elements, so a generic description overrode per-page values. Resolved by updating the element directly rather than rendering a second one.               |
| Policy documents named a non-existent contact address | Review               | The privacy policy and terms directed data-subject requests to an address on a domain not under the operator's control, so every such request would have failed silently. Corrected to the operational address.  |
| Cookie policy inaccurate after a feature addition     | Review               | The language toggle introduced a browser storage key that the policy's enumeration of stored items did not list, rendering the published policy incorrect about the system's own behaviour. The entry was added. |

**Table 6.3** — Defects found during development, with detection mechanism and resolution.

Two observations follow from this table. First, the defects that reached a user-visible state were predominantly those for which no automated signal existed: the two server errors and the layout fault were all found by a person exercising the interface. Second, where a mechanical check did exist — the build, the linter, the locale checker — it caught the fault before deployment in every instance. This is the empirical basis for the recommendation in Section 8.3 that an automated suite is the highest-value next investment.

## 6.6 Threats to validity

- **No formal user evaluation.** No usability study, questionnaire or controlled trial was conducted with the intended student population. Claims about usability rest on adherence to accepted design criteria such as WCAG contrast thresholds, not on measured user performance or reported satisfaction. No claim of demonstrated user acceptance is made anywhere in this report.
- **Single-operator testing.** Payment flows were exercised with a live MTN Mobile Money account. The Orange Money path shares the same adapter, normalisation and settlement logic and differs only in the operator identifier transmitted, but it was not exercised end-to-end with a live Orange subscription. It is therefore verified by construction rather than by observation.

- **No load testing.** The system has not been subjected to concurrent load. Statements about scalability are architectural arguments, not measurements.
- **Single-tester bias.** All manual testing was performed by the developer, who knows the intended paths. This systematically under-samples the unexpected sequences that unfamiliar users generate.
- **Limited device coverage.** Responsive behaviour was verified by viewport emulation and on a limited set of physical devices, not across the range of handsets in actual use.

# Results and Discussion

## 7.1 Objectives achieved

| OBJ. | STATUS             | EVIDENCE                                                                                                                                                                                                             |
|------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O1   | Achieved           | Requirements analysis and comparative review of five classes of existing system (Chapter 2, Table 2.1), yielding 22 functional and 14 non-functional requirements.                                                   |
| O2   | Achieved           | Three-layer architecture (Figure 4.1) with a 25-table normalised schema under version-controlled migration.                                                                                                          |
| O3   | Achieved           | Groups with real-time messaging, note repository and Q&A forum with voting, accepted answers and reputation, implemented and deployed.                                                                               |
| O4   | Achieved           | Tutor applications, administrative review, weekly availability and booking, with prepaid session packs.                                                                                                              |
| O5   | Achieved           | Mobile money integration with idempotent granting and background reconciliation; verified by T-08 through T-10.                                                                                                      |
| O6   | Achieved           | Authenticated delivery with signed, expiring URLs issued only after an entitlement check; verified by T-11.                                                                                                          |
| O7   | Achieved           | Progressive Web Application with per-resource caching, transport-failure recovery and mobile-first layout; verified by T-17 and T-18.                                                                                |
| O8   | Achieved           | 1,228 verified keys per language with automated parity checking; verified by T-15 and T-16.                                                                                                                          |
| O9   | Partially achieved | The system is publicly deployed and validated against the stated requirements by structured manual testing. Validation did not include a formal user evaluation, which is why this objective is not claimed in full. |

**Table 7.1** — Attainment of objectives against evidence.

## 7.2 Measured outcomes

| MEASURE              | VALUE          | NOTE                                                |
|----------------------|----------------|-----------------------------------------------------|
| Source lines, client | approx. 14,700 | 26 route components, 17 shared primitives, 6 hooks. |
| Source lines, server | approx. 6,000  | 12 route modules, 7 services, 7 middleware.         |

| MEASURE                       | VALUE                  | NOTE                                                                                                             |
|-------------------------------|------------------------|------------------------------------------------------------------------------------------------------------------|
| REST endpoints                | 88                     | Enumerated in Appendix A.                                                                                        |
| Database tables               | 25                     | Across 6 applied migrations.                                                                                     |
| Translation keys per language | 1,228                  | Parity verified automatically; no missing, extra or mismatched keys.                                             |
| Note download, 3.19 MB file   | 300 s → 31 s           | Before and after CDN redirect, same file and connection. The prior figure is a client timeout, not a completion. |
| Version control history       | 65 commits             | 27 July to 11 September 2026.                                                                                    |
| Static analysis baseline      | 42 errors, 13 warnings | Held constant; no change permitted to increase it.                                                               |

**Table 7.2** — Measured characteristics of the delivered system. All figures are counted from the repository at the final commit; none are estimates.

## 7.3 Discussion of design decisions

**Treating the browser as an observer of payment.** The most consequential decision in the system was to deny the client any authority over payment outcomes. A design in which the browser reports success, or in which the poll is the sole settlement path, is simpler and works under demonstration conditions. It fails in exactly the circumstances the target users experience most often: a dropped connection, a closed tab, a PIN entered after a delay. Moving the authority to a server-side reconciler cost additional complexity and a background process, and it is the reason a student who loses signal mid-payment still receives what they paid for.

**Computing trust rather than storing it.** Deriving publication privileges from recent review history, instead of persisting a flag, eliminates a class of failure in which a privilege outlives the conduct that justified it. The general principle — that derived state cannot drift out of step with the facts it is derived from, whereas stored state requires discipline to maintain — applied in several places in the design.

**Separating stored values from displayed labels.** The internationalisation rule that persisted values remain in English proved more consequential than anticipated. It is the difference between a bilingual interface and a partitioned platform in which francophone and anglophone users cannot find one another's content. The failure mode would have been silent: no error, simply empty search results for some pairs of users.

**Designing against WhatsApp rather than against web platforms.** Because the incumbent is a free, universally installed messaging application, the acceptance threshold is set by it rather than by comparable web systems. This justified investment in installability, offline tolerance and first-load behaviour that would be difficult to defend against a purely feature-based comparison.

## **7.4 Constraints imposed by context**

Several characteristics of the delivered system are direct consequences of operating on free-tier infrastructure, and would not be present in a resourced deployment. The application server suspends when idle, necessitating the retry and notification behaviour of Section 5.7. Connection pooling limits dictated the dual database connection strategy of Section 4.3.2. The ephemeral filesystem of the hosting platform required that all uploads be externalised, which in turn made the CDN redirect of Section 5.3 both possible and necessary. These are not design preferences; they are adaptations, and they are documented as such so that a future maintainer does not mistake them for arbitrary complexity and remove them.

# Conclusion and Future Work

---

## 8.1 Conclusion

This project set out to address the absence of a single accessible platform on which university students could share academic resources, resolve specific difficulties and obtain paid peer tutoring, using payment instruments appropriate to the Cameroonian context. A full-stack web application meeting that description was designed, implemented and publicly deployed.

Eight of the nine stated objectives were achieved in full and the ninth in part, the shortfall being the absence of a formal user evaluation rather than any deficiency in the delivered functionality. The system integrates four capabilities that are elsewhere available only separately — collaborative groups, a shared content repository, a reputation-bearing question forum, and a tutoring marketplace — and settles payment through the mobile money instruments the intended users actually hold.

Beyond the functional result, the project produced engineering outcomes of wider applicability. The payment design demonstrates that reliable entitlement granting over an asynchronous and unreliable settlement rail is achievable through server-side price resolution, idempotent granting and background reconciliation, without requiring the client to remain connected. The content-delivery design shows that protected digital goods can be served securely from constrained infrastructure by separating the authorisation decision from the byte delivery, with a measured order-of-magnitude improvement in transfer time for unprotected content. The internationalisation work establishes a rule — that persisted values remain canonical while only labels are translated — whose violation would have silently partitioned the user base along linguistic lines.

The project also illustrates, through the defect record of Table 6.3, the practical value of mechanical verification. Every fault for which an automated check existed was caught before deployment; every fault that reached a user-visible state did so in an area where no such check existed.

## 8.2 Limitations

The following limitations are stated so that the results are not read as claiming more than was demonstrated.

**Methodological**

- Requirements were reasoned from observation and comparative analysis rather than elicited through a formal instrument administered to the user population.
- No user evaluation was conducted; no claim of measured usability or user acceptance is made.
- All testing was performed by the developer, with the sampling bias that implies.

**Technical**

- There is no automated regression test suite.
- Sessions are stateless tokens, so a token remains valid until expiry even after sign-out or suspension. A short lifetime bounds the exposure; it does not remove it.
- The static analysis baseline of 42 errors was held constant rather than eliminated.
- The system has not been load tested; scalability claims are architectural rather than measured.
- The Orange Money path is verified by construction rather than by live end-to-end execution.

**Functional**

- Direct messaging to a tutor persists to browser storage only and is not transmitted; the feature is present in the interface but not functional end-to-end.
- Session resource downloads on the tutor profile are simulated rather than served.
- Notification preferences in settings are not persisted.
- Email notification is not implemented; notifications are in-application only.
- Server-composed text — notification bodies, certain payment messages and the rendered receipt — remains English-only, so translation coverage is complete for the interface but not for the system as a whole.
- Two landing-page figures are placeholder values rather than measured statistics and should be connected to the live counters or removed before further promotion.

**8.3 Recommendations for future work**

| RANK | RECOMMENDATION                    | RATIONALE                                                                                                                                                                                                                |
|------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Establish an automated test suite | The evidence of Table 6.3 is that faults escaped precisely where no mechanical check existed. Priority should follow risk: entitlement granting and reconciliation first, then authorisation, then interface regression. |

| RANK | RECOMMENDATION                       | RATIONALE                                                                                                                                                       |
|------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2    | Conduct a formal user evaluation     | A study with the intended population would convert reasoned usability claims into measured ones, and would surface requirements that observation alone did not. |
| 3    | Complete the server-side translation | Notification and receipt text is the remaining English-only surface, and it reaches users who selected French.                                                  |
| 4    | Implement tutor messaging properly   | The capability is advertised in the interface but does not function, which is worse for trust than its absence.                                                 |
| 5    | Introduce token revocation           | A short-lived access token with a refresh mechanism, or a revocation list, would close the window in which a suspended account retains access.                  |
| 6    | Conduct load testing                 | Required before any claim about concurrent capacity, and before migrating off free-tier infrastructure on the basis of measured need.                           |
| 7    | Add email notification               | In-application notification is insufficient for events the user must act upon while not using the platform, such as a booking request.                          |
| 8    | Extend payment instruments           | Additional local rails would widen access, and a wallet abstraction would reduce the coupling to a single aggregator.                                           |

**Table 8.1** — Recommendations for further work, in priority order.

# References

---

## NOTE ON SOURCES

This report is primarily an engineering artefact, and the majority of its sources are the official specifications and documentation of the technologies employed. The two pedagogical references below should be verified against a library catalogue before submission, as bibliographic details were not confirmed against an indexing database during preparation.

## Pedagogy

- [1] Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press.
- [2] Topping, K. J. (2005). Trends in peer learning. *Educational Psychology*, 25(6), 631–645.

## Standards and specifications

- [3] World Wide Web Consortium (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation. Available at: <https://www.w3.org/TR/WCAG21/>
- [4] Jones, M., Bradley, J. and Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] Provos, N. and Mazières, D. (1999). A Future-Adaptable Password Scheme. In *Proceedings of the USENIX Annual Technical Conference*.
- [6] OWASP Foundation. *OWASP Top Ten Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/>
- [7] European Parliament and Council (2016). *Regulation (EU) 2016/679 — General Data Protection Regulation*. Official Journal of the European Union.
- [8] World Wide Web Consortium. *Web Application Manifest*. W3C Working Draft. Available at: <https://www.w3.org/TR/appmanifest/>

## Technical documentation

- [9] Meta Open Source. *React Documentation*. Available at: <https://react.dev/>
- [10] Vite. *Vite Documentation*. Available at: <https://vite.dev/>
- [11] OpenJS Foundation. *Express Documentation*. Available at: <https://expressjs.com/>
- [12] Prisma Data, Inc. *Prisma ORM Documentation*. Available at: <https://www.prisma.io/docs>

- [13] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available at: <https://www.postgresql.org/docs/>
- [14] Tailwind Labs. *Tailwind CSS Documentation*. Available at: <https://tailwindcss.com/docs>
- [15] Socket.IO. *Socket.IO Documentation*. Available at: <https://socket.io/docs/>
- [16] Cloudinary Ltd. *Cloudinary Documentation*. Available at: <https://cloudinary.com/documentation>
- [17] Hachther LLC. *MeSomb Developer Documentation*. Available at: <https://mesomb.hachther.com/en/api/v1.1/>
- [18] Google. *Workbox Documentation*. Available at: <https://developer.chrome.com/docs/workbox/>
- [19] i18next. *i18next Documentation*. Available at: <https://www.i18next.com/>

## Project artefacts

- [20] Tazanu, S. (2026). *StudyHub source repository*. Available at: <https://github.com/Tazanu/studyhubwebapp>
- [21] Tazanu, S. (2026). *StudyHub deployed application*. Available at: <https://studyhubwebapp.vercel.app>
- [22] Tazanu, S. (2026). *StudyHub Technical Documentation*. Repository path: `docs/StudyHub-Technical-Documentation.pdf`

# API Endpoint Summary

The API exposes 88 endpoints across twelve modules, all prefixed `/api` except the health probe. A complete per-endpoint reference is provided in the accompanying technical documentation [22]; this appendix summarises the surface.

| MODULE                      | ENDPOINTS | RESPONSIBILITY                                                                                 |
|-----------------------------|-----------|------------------------------------------------------------------------------------------------|
| <code>/auth</code>          | 4         | Registration, authentication, current identity, password change.                               |
| <code>/groups</code>        | 21        | Group lifecycle, membership, join requests, messaging, reactions, typing state, read tracking. |
| <code>/notes</code>         | 6         | Listing, upload, download accounting, gated file retrieval, deletion.                          |
| <code>/qa</code>            | 14        | Questions, answers, voting, acceptance, bookmarks, follows, comments, categories.              |
| <code>/tutors</code>        | 13        | Directory, applications, availability, bookings, session credits.                              |
| <code>/premium</code>       | 10        | Paid catalogue, purchase, cancellation, receipts, entitlement checks, publishing status.       |
| <code>/payments</code>      | 3         | Generic initiation, status polling, transaction history.                                       |
| <code>/admin</code>         | 14        | Statistics, user administration, tutor review, paid-note moderation, payment diagnostics.      |
| <code>/users</code>         | 5         | Public profiles, contribution statistics, leaderboard, profile maintenance.                    |
| <code>/notifications</code> | 3         | Retrieval and read-state management.                                                           |
| <code>/contact</code>       | 3         | Public submission and administrative inbox.                                                    |
| <code>/stats</code>         | 1         | Public platform counters.                                                                      |
| <code>/health</code>        | 1         | Liveness probe; unprefixed and unrestricted.                                                   |

Table A.1 — API surface by module.

## A.2 Real-time event vocabulary

| EVENT                                                   | DIRECTION        | MEANING                                                             |
|---------------------------------------------------------|------------------|---------------------------------------------------------------------|
| <code>group:join</code>                                 | client to server | Request admission to a group room; membership verified server-side. |
| <code>group:leave</code>                                | client to server | Depart a room.                                                      |
| <code>typing:start</code> ,<br><code>typing:stop</code> | client to server | Composition state.                                                  |
| <code>group:joined</code>                               | server to client | Admission confirmed.                                                |
| <code>message:new</code>                                | server to room   | A message was posted.                                               |
| <code>message:edit</code>                               | server to room   | A message was amended within the permitted window.                  |
| <code>message:delete</code>                             | server to room   | A message was soft-deleted.                                         |
| <code>message:reaction</code>                           | server to room   | Reactions on a message changed.                                     |
| <code>typing:update</code>                              | server to room   | Current set of composing users.                                     |
| <code>user:joined</code> ,<br><code>user:left</code>    | server to room   | Presence change.                                                    |

**Table A.2** — Socket.IO event vocabulary.

# Database Schema Summary

| TABLE                                      | PURPOSE                                                                                        |
|--------------------------------------------|------------------------------------------------------------------------------------------------|
| users                                      | Identity, credentials, profile, role, reputation, active state.                                |
| groups                                     | Study group definition, subject and join policy.                                               |
| user_groups                                | Membership, resolving the many-to-many relation and carrying in-group role.                    |
| group_join_requests                        | Pending admissions to approval-gated groups.                                                   |
| group_read_status                          | Per-user read position, supporting unread indicators.                                          |
| group_messages                             | Messages with optional attachment, reply reference, edit and soft-delete flags.                |
| group_message_reactions                    | Emoji reactions attributed to a user and message.                                              |
| notes                                      | Uploaded material with subject, tags, file reference, download count, premium flag and price.  |
| questions                                  | Forum questions with subject, category, tags, optional audio and images, view and vote counts. |
| answers                                    | Answers with optional audio and images, vote count and acceptance flag.                        |
| question_votes ,<br>answer_votes           | Individual votes, uniquely constrained per voter and target.                                   |
| question_bookmarks ,<br>question_followers | Personal saving and subscription to activity.                                                  |
| answer_comments                            | Comments attached to answers.                                                                  |
| tutors                                     | Tutor extension of a user: subjects, biography, experience, rate, application status.          |
| tutor_availability                         | Weekly recurring availability slots.                                                           |
| bookings                                   | Sessions booked between a student and tutor, with date, times, amount and status.              |
| session_credits                            | Prepaid session packs and remaining balance.                                                   |

| TABLE                 | PURPOSE                                                                      |
|-----------------------|------------------------------------------------------------------------------|
| transactions          | Payment ledger: amount, operator, payer, provider reference, status.         |
| premium_notes         | Paid material with price, review state, quality report and publication flag. |
| purchased_notes       | Entitlement record consulted before any protected file is released.          |
| premium_subscriptions | Publisher subscription periods.                                              |
| notifications         | In-application notifications with type, recipient and read state.            |
| contact_messages      | Public contact submissions with read state.                                  |

**Table B.1** — Database tables and their purpose. Twenty-five tables in total; vote and bookmark tables are grouped above for brevity.

# Configuration Reference

The system is configured entirely through environment variables; no credential is committed to version control. Variable *names* only are reproduced here.

## C.1 Application server

| VARIABLE                                                                   | REQUIRED | PURPOSE                                                                         |
|----------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------|
| DATABASE_URL                                                               | Yes      | Pooled database connection used by the running application.                     |
| DIRECT_URL                                                                 | Yes      | Direct database connection, used only for migrations.                           |
| JWT_SECRET                                                                 | Yes      | Token signing secret.                                                           |
| JWT_EXPIRES_IN                                                             | No       | Token lifetime; defaults to 24 hours.                                           |
| BCRYPT_ROUNDS                                                              | No       | Password hashing work factor; defaults to 12.                                   |
| ALLOWED_ORIGINS                                                            | Yes      | Comma-separated permitted client origins.                                       |
| FRONTEND_URL                                                               | Fallback | Used when the allow-list is unset.                                              |
| CLOUDINARY_CLOUD_NAME ,<br>CLOUDINARY_API_KEY ,<br>CLOUDINARY_API_SECRET   | Yes      | Object storage credentials.                                                     |
| MESOMB_APPLICATION_KEY ,<br>MESOMB_ACCESS_KEY ,<br>MESOMB_SECRET_KEY       | Yes      | Payment provider credentials, verified read-only at startup.                    |
| NODE_ENV                                                                   | Yes      | Must be <code>production</code> on a deployed server; gates the HTTPS redirect. |
| PORT                                                                       | No       | Supplied by the platform; defaults to 5000 locally.                             |
| RATE_*                                                                     | No       | Five window and maximum pairs overriding the rate limits.                       |
| RECONCILE_INTERVAL_MS ,<br>RECONCILE_LOOKBACK_MS ,<br>RECONCILE_ABANDON_MS | No       | Settlement sweep timing.                                                        |

## C.2 Client

| VARIABLE              | REQUIRED    | PURPOSE                                                                  |
|-----------------------|-------------|--------------------------------------------------------------------------|
| VITE_API_URL          | Yes         | API base address. Inlined at build time; a change requires redeployment. |
| VITE_SITE_URL         | Recommended | Public origin, used to generate absolute sitemap and canonical URLs.     |
| VITE_PLAUSIBLE_DOMAIN | No          | Enables analytics; omission disables it cleanly.                         |

### DEPLOYMENT CAUTION

Client variables are compiled into the bundle at build time rather than read at run time. Altering one in a hosting dashboard has no effect until the project is rebuilt. An incorrect API address therefore produces a client that loads perfectly and then fails every request, presenting as a server outage when it is a build configuration fault.

End of report. Prepared from the project repository at commit b6e46f4. No credential values appear in this document.